DATA ENGINEERING COURSE

Phase 1: Project Proposal & Environment Setup

Project: **Chicago Crimes - End-to-End Data Pipeline**

Submission deadline: Thursday, 12 Nov 2026, 11:59 PM

1. Team Members

#	Full name	Student ID	Assigned role
1	Amr Khaled Hammad	692300242	Ingestion & Python cleaning
2	Mostafa Ibrahim Mohamed	692300021	Database design & SQL (star schema, DDL, views)
3	Sama Hesham Abbas	692300386	Data quality checks & validation
4	Mohamed Osama Hassan	692300070	Kestra orchestration & reproducibility (Docker)
5	Mohamed Abdelhakem Abdelaal	692300390	Analytics & dashboard

Roles are based on the suggested split in the project brief (database and quality shown separately for a 5-member team). Roles are not silos: every member is expected to understand the whole workflow.

DATA LINK : https://catalog.data.gov/dataset/crimes-one-year-prior-to-present

2. Problem Statement

Public crime records are published as large flat files that are hard to analyse directly. Our goal is to build a reproducible pipeline that loads the Chicago crime incident records into PostgreSQL, cleans and models them (star schema), validates their quality, automates the workflow with Kestra, and exposes the results in an interactive dashboard. The dashboard will help answer when, where and what kind of crime happens in Chicago and how often incidents end in an arrest.

3. Dataset Overview

Item	Details
Dataset	Chicago Crimes (reported incidents recorded by the Chicago Police Department)
Publisher	City of Chicago Data Portal (public open data). Exact dataset URL: **[team to paste link]**
Format / file	CSV, stored in the repo at `data/chicago_crimes.csv` (about 49 MB)
Size	**230,425 rows x 17 columns** (meets the 10,000-row target)
Time coverage	4 Nov 2025 to 3 Nov 2026 (about one year; Nov 2026 is a partial month with 1,239 rows)
Privacy	No names or personal identifiers. Locations are block-level addresses published by the city.
Instructor approval	[Pending / Approved on____] - must be confirmed before Phase 2

3.1 Columns, data types and target types

#	Source column	Source type	Target name / PostgreSQL type	Notes
1	CASE#	text	case_number VARCHAR(12)	Not unique (25 repeated values, e.g. multi-victim homicides)
2	DATE OF OCCURRENCE	text (MM/DD/YYYY hh:mm:ss AM/PM)	occurred_at TIMESTAMP	Header has a double space; parse with explicit format

3	BLOCK	text	block VARCHAR(60)	Anonymised block address
4	IUCR	text	iucr CHAR(4)	Leading space in header; keep as text (leading zeros)
#	**Source column**	**Source type**	**Target name / PostgreSQL type**	**Notes**
5	PRIMARY DESCRIPTION	text	primary_type VARCHAR(50)	30 distinct values
6	SECONDARY DESCRIPTION	text	secondary_type VARCHAR(80)	Sub-category of primary type
7	LOCATION DESCRIPTION	text	location_description VARCHAR(80)	122 distinct; 1,114 nulls
8	ARREST	text (Y/N)	arrest BOOLEAN	15.5% Y
9	DOMESTIC	text (Y/N)	domestic BOOLEAN	19.2% Y
10	BEAT	integer	beat SMALLINT	275 distinct police beats
11	WARD	integer	ward SMALLINT	398 rows have ward = 0 (invalid, valid range 1-50)
12	FBI CD	text	fbi_code VARCHAR(4)	Keep as text (e.g. '07', '08B'); 26 distinct
13	X COORDINATE	float	x_coord NUMERIC	457 nulls
14	Y COORDINATE	float	y_coord NUMERIC	457 nulls
15	LATITUDE	float	latitude NUMERIC(9,6)	457 nulls
16	LONGITUDE	float	longitude NUMERIC(9,6)	457 nulls
17	LOCATION	text '(lat, lon)'	dropped (redundant)	Derivable from latitude/longitude

3.2 Known limitations (found by a first profiling pass)

- **Missing values:** 1,114 rows without location description; 457 rows without coordinates.
- **Non-unique CASE#:** 25 case numbers repeat (with no fully duplicated rows). A surrogate key will be used in the fact table.
- **Invalid ward:** 398 rows have ward 0 and will be flagged or set to NULL during cleaning.
- **Messy headers:** double space and leading spaces in column names; will be standardised to snake_case.
- **Rolling window:** the portal publishes roughly the last year, so a re-download gives a different date range. We freeze this CSV as our reference snapshot.
- **Reported crimes only:** counts reflect police reports, not all crime. Insights must be worded accordingly.

4. Analytical Questions

#	Question	Fields and aggregation
Q1	Which crime types have the highest arrest rate, and how does the arrest rate for the top types change from month to month?	primary_type, arrest, occurred_at (month). Arrest rate = arrests / incidents per type per month; ranked, with a minimum-volume filter.
Q2	At what hours of the day and on which days of the week do the main crime categories peak, and how do weekdays compare with weekends?	occurred_at (hour, weekday/weekend flag), primary_type. Counts and share of total per hour x day-of-week (heatmap).
Q3	Which 10 wards have the most incidents, and how do their domestic-incident share and most common location types differ?	ward, domestic, location_description. Count per ward, domestic % and top location type per ward (window functions).

All three need multi-field grouping and derived measures, and will be implemented as SQL views in Phase 3.

5. Proposed Architecture

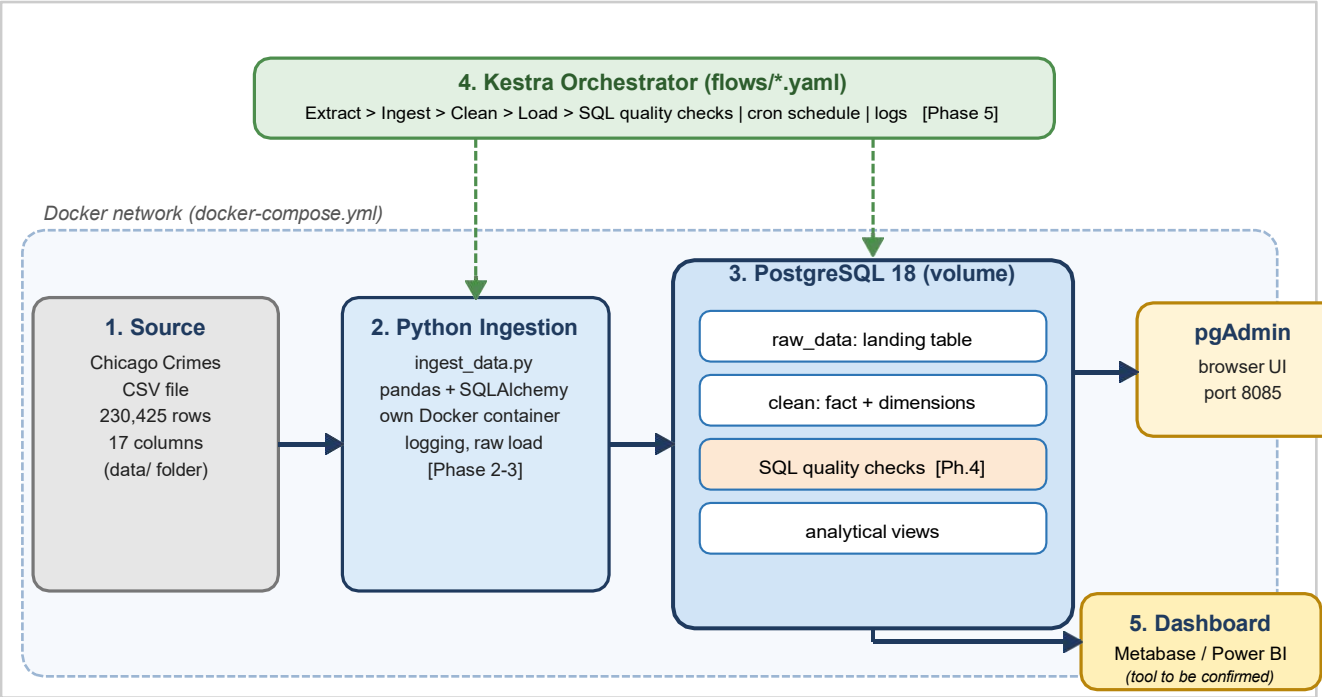


Figure 1: planned data flow. Solid arrows = data, dashed green arrows = orchestration control. The dashboard tool is still to be confirmed; the diagram will be updated to match what is actually implemented.

Layer	Technology	Responsibility	Phase
Source	CSV file from City of Chicago	Raw snapshot, versioned in the repo or documented download link	1
Ingestion	Python (pandas, SQLAlchemy, psycopg) in Docker	Read file, log counts and timestamps, load untouched into raw_data schema	2
Cleaning & modelling	Python + SQL DDL	Types, nulls, duplicates, star schema with PK/FK and indexes	3
Quality	Python pre-load + SQL post-load checks	PK uniqueness, FK integrity, null thresholds, ranges, row counts; halt on failure	4
Orchestration	Kestra	End-to-end flow, idempotent re-runs, cron schedule, monitoring	5
Serving	Dashboard tool (to be confirmed)	KPI cards, charts, filters reconciled with SQL results	6

6. Expected Database Tables

Schema.table	Main columns	Purpose
raw_data.chicago_crimes_raw	all 17 source columns as TEXT + ingested_at, source_file	Untouched landing copy for traceability
clean.fact_crime	crime_id (PK, surrogate), case_number, occurred_at, date_key (FK), crime_type_key (FK), location_type_key (FK), area_key (FK), block, arrest, domestic, latitude, longitude	One row per reported incident
clean.dim_date	date_key (PK), full_date, year, quarter, month, day, day_of_week, is_weekend	Calendar attributes for time analysis
clean.dim_crime_type	crime_type_key (PK), iucr, primary_type, secondary_type, fbi_code	Crime classification
clean.dim_location_type	location_type_key (PK), location_description	Where the incident happened (street, apartment...)
clean.dim_area	area_key (PK), beat, ward	Police beat and ward

Schema.table	Main columns	Purpose
analytics.vw_arrest_rate_by_type_month	primary_type, month, incidents, arrests, arrest_rate	Answers Q1
analytics.vw_crime_by_hour_dow	day_of_week, hour, primary_type, incidents	Answers Q2
analytics.vw_ward_summary	ward, incidents, domestic_pct, top_location_type	Answers Q3

The hour of day is derived from occurred_at at query time. The final model may change after Phase 3; this is the initial design.

7. Repository and Environment Setup

GitHub repository: **https://github.com/mohamedabdelhakam665/Data-Engineering-Project**

```
|--- docs/                  # proposal, architecture diagram, data dictionary
|--- scripts/               # ingest_data.py, transform_data.py
|--- sql/                   # DDL, quality checks, views
|--- flows/                 # Kestra YAML flows
|--- data/                  # source CSV (documented in README)
|--- docker-compose.yml     # PostgreSQL 18 + pgAdmin (persistent volumes)
|--- .env.example           # variables without real passwords
|--- .gitignore
`--- README.md              # how to run
```

- **docker-compose.yml** starts PostgreSQL 18 (database chicago_crimes, port 5432) and pgAdmin (port 8085), each with a named volume so data survives `docker compose down`.
- Credentials are read from a local `.env` file (not committed); `.env.example` documents the variables.
- Inside the Docker network, pgAdmin and the ingestion container reach PostgreSQL by its service name (`pgdatabase`), not by localhost.
- Each member runs `docker compose up -d` on their own laptop and confirms the login to pgAdmin (screenshots in the repo under docs/).

8. AI Usage Disclosure

As required by the project brief, we document how AI tools were used. **Team to complete honestly before submission:** an AI assistant (Claude) was used to help draft this proposal, the first data profiling, the folder structure and the README/compose template. The team must review every part, run the setup themselves, and be able to explain the dataset, questions, architecture and docker-compose file individually.

Part	AI-assisted?	Reviewed and understood by
Proposal text and architecture diagram	Yes (draft)	All members
Dataset profiling numbers	Yes (script) - re-run to verify	All members
docker-compose.yml / .env / README	Yes (template)	All members